

Independent cross-check of quantum LDPC code distances against the `codeDistance` package, and a route to distance bounds for larger codes

Juan Cruz-Benito

June 10, 2026

Abstract

We cross-checked the code-distance values recorded by our discovery pipeline [1] (BP-OSD plus a SciPy/HiGHS MILP) against the independent `codeDistance` package of Webster, Jacob, and Higgott (arXiv:2603.22532). Across 198 distinct codes (43 CSS, 155 non-CSS perturbed-bicycle), independent methods matched or corroborated our recorded values with *zero contradictions*, including the codes where BP-OSD overestimated the distance by up to $12\times$. We then show, on Webster’s own evolutionary search `QDistEvol`, that a cheap heuristic matches the recorded MILP value on every one of the worst overestimate cases and even tightens one of our incumbents. Finally we analyse what these methods can and cannot do for *larger* codes, where the MILP times out, and give a concrete bound-bracketing workflow that combines a heuristic upper bound with the certified lower bound HiGHS can report but our current pipeline does not persist.

1 Setup and protocol

The codes under test are the bivariate-bicycle (BB) and perturbed-bivariate-bicycle (PBB) quantum LDPC codes discovered in [1], whose distances were recorded there via BP-OSD and a SciPy/HiGHS MILP (catalogs `ilp_catalog.json` for CSS and the `campaign-7` record for non-CSS).

The `codeDistance` package exposes `CSScodeDistance` (CSS) and `codeDistance` (non-CSS, symplectic two-block input). We drive four methods: `BZDistMW` (Brouwer–Zimmermann enumeration, exact on completion), `MIPDist` (OR-Tools/SCIP), `QDistEvol` (evolutionary heuristic), and `decoderDist` (BP-OSD via the `ldpc` package).

The protocol rests on one principle: **trust the returned witness, not the reported distance**. For every run we recompute the weight of the returned operator (Hamming for CSS, symplectic for non-CSS) and verify that it commutes with all checks and anticommutes with at least one logical. A *verified* witness of weight w is then a hard certificate that $d \leq w$, independent of the method and of solver bookkeeping. Two consequences shape the reading of every result below:

- `MIPDist` is always recorded as an *incumbent*: the public package result does not expose OR-Tools’ optimality status, so we never promote a finished solve to a proof. `BZDistMW` is `proven_exact` only when it runs to completion.
- The highest-impact signal is a verified witness *lighter* than a distance we claim exact (the `CRITICAL_LOWER` verdict). None was observed.

2 Results

Overall. 198 distinct codes, 271 method-runs (254 MIPDist incumbents, 17 BZDistMW proofs), every returned witness verified. Table 1 gives the per-code verdict.

verdict	codes	meaning
agree	196	min verified witness weight = our recorded value
webster_higher	2	heavier valid logical (looser bound), not a contradiction
CRITICAL	0	no verified witness below a claimed-exact distance

Table 1: Per-code verdicts over the 198-code cross-check. The two **webster_higher** rows are $n=108$, $d=10$ codes whose MIP returned weight 12 even at a 600s budget ($12 \geq 10$, consistent).

CSS (Comparison A). All 43 CSS codes agree: the two Bravyi baselines ($\llbracket 72, 12, 6 \rrbracket$, $\llbracket 144, 12, 12 \rrbracket$), 32 catalog rows at $n=144$, and the 9 “BP-OSD overestimated” rows at $n=288/360$. We stress that `ilp_catalog.json` labels are not solver certificates; the non-Bravyi rows are *corroborated* by independent witnesses, not upgraded to proven-exact.

The headline cross-check is the overestimate subset (Table 2): these are the codes where our pipeline most aggressively corrected BP-OSD. An independent MIP (OR-Tools/SCIP, distinct from our SciPy/HiGHS formulation) returned a verified logical at our recorded value in both X and Z on all nine. Because a verified weight- w witness certifies $d \leq w$, this *proves* the upper bounds that refute the BP-OSD estimates—e.g. $\llbracket 360, 40 \rrbracket$ with a weight-2 witness against a BP-OSD estimate of 24, a $12\times$ overestimate. Equality with our recorded value is corroboration, not an independent optimality proof (all runs are incumbents, and these catalog rows are incumbents too); no lighter witness was ever found.

code	BP-OSD d	MILP / verified witness	BP-OSD / witness
$\llbracket 360, 40 \rrbracket$	24	2	12.0 $\times$
$\llbracket 360, 40 \rrbracket$	22	2	11.0 $\times$
$\llbracket 360, 40 \rrbracket$	20	2	10.0 $\times$
$\llbracket 288, 32 \rrbracket$	22	4	5.5 $\times$
$\llbracket 360, 32 \rrbracket$	20	4	5.0 $\times$
$\llbracket 360, 32 \rrbracket$	16	4	4.0 $\times$
$\llbracket 288, 32 \rrbracket$	20	6	3.3 $\times$
$\llbracket 288, 32 \rrbracket$	12	4	3.0 $\times$
$\llbracket 288, 24 \rrbracket$	18	12	1.5 $\times$

Table 2: The nine “BP-OSD overestimated” CSS codes. The verified witness weight matches our recorded MILP value on all nine, certifying the corrected (much lower) upper bound against BP-OSD overestimates of up to $12\times$.

Non-CSS (Comparison C). Of 155 exact non-CSS codes ($n \leq 180$, $d > 6$, plus the $n=36$ baselines), 153 agree and 2 are looser; none contradicts. The non-CSS symplectic linearization our MILP uses therefore held on every code tested: every returned witness was a valid logical and none was lighter than our recorded distance. A targeted 600s rerun of an initial 13 looser incumbents closed 11 of them to agreement with our exact record.

Heuristics (Comparison B): does QDistEvol close the BP-OSD gap? On the 17 worst overestimate codes (the 9 CSS rows plus the 8 largest non-CSS $d_{\text{BP-OSD}} - d_{\text{MILP}}$ gaps) we ran QDistEvol and decoderDist and compared each verified witness weight to our recorded MILP value (Table 3). QDistEvol reached the recorded MILP value on *all 17*, and on one non-CSS code ([360, 12], BLISS c447dcb2d101746f) it *improved* our incumbent, finding a weight-20 logical where the recorded value was 24. It is cheap ($\sim 46\text{--}133\text{s/code}$).

Two nuances sharpen the BP-OSD story. On CSS, decoderDist (a well-run BP-OSD) also matches the recorded `ilp_d` value on all nine—so the overestimates recorded in our catalog reflect *our pipeline’s* BP-OSD configuration, not an intrinsic BP-OSD limitation. On non-CSS, decoderDist timed out with no verified logical on all eight; QDistEvol was the only Tier-2 heuristic that returned valid witnesses there. This is exactly the regime where a heuristic alternative to BP-OSD matters.

set	decoderDist reaches recorded MILP	QDistEvol reaches recorded MILP
CSS (9)	9/9	9/9
non-CSS (8)	0/8 (timed out, no logical)	8/8
total	9/17	17/17

Table 3: Gap-closing on the worst BP-OSD-overestimate codes.

3 Distance bounds for larger codes

The pressing practical problem is that for most $n \geq 108$ codes the MILP times out before proving optimality. We separate two things a timeout costs us, because the available remedies differ:

upper bound: $d \leq U$ (a low-weight logical), lower bound: $d \geq L$ (a proof nothing is lighter).

Upper bounds are sound at any size; the limit is tightness. Since we verify every witness, “method returned a weight- w logical $\Rightarrow d \leq w$ ” holds for arbitrarily large codes—the verification is GF(2) linear algebra. What degrades with size is whether a method *finds* a tight witness. Here QDistEvol genuinely helps: it is cheap, independent, and on our hardest non-CSS codes it matched the MILP and even beat an incumbent (Table 3). MIPDist does *not* break the wall—it is the same NP-hard problem in a different solver and returned only incumbents on the hard codes—so its value at scale is a tight *witness*, not a certificate.

The lower bound is not being persisted. Certifying $d = w$ needs a lower bound. `scipy.optimize.milp` returns `mip_dual_bound` and `mip_gap` (confirmed in SciPy 1.17)—a *certified* lower bound on each per-logical objective, valid even on timeout—but the current pipeline keeps only the incumbent objective and the proven-optimal flag. In a future non-early-exit pass, capturing $L_i = \lfloor b_i \rfloor$ for every relevant logical subproblem, where b_i is that subproblem’s `mip_dual_bound`, gives

$$\min_i L_i \leq d \leq U,$$

where U is the best verified witness from MILP, QDistEvol, BP-OSD, or another method. If every subproblem has a valid dual bound and $\min_i L_i = U$, the distance is certified exact. If some subproblems are skipped or never started, the global lower bound is still incomplete, so those cases should be reported as partially bracketed rather than exact.

Closing more codes. To actually prove exactness on borderline codes:

1. **Use the heuristic witness as a cutoff target.** The current SciPy `milp` wrapper does not expose a MIP-start argument, but a verified `QDistEvol` witness still gives the target U : the MILP/SAT task becomes “prove no logical of weight $< U$ ” (or raise the dual bound to U). A backend with true MIP starts could additionally inject the witness as the initial incumbent.
2. **Stronger solver:** if licensed, a direct Gurobi-backed run (e.g. a Webster/Gurobi method or our own Gurobi formulation) may be faster than HiGHS/SCIP on these instances.
3. **SAT (`pySATDist`):** a different paradigm in which “no logical of weight $< U$ ” returning UNSAT is itself a certified lower bound; SAT sometimes closes instances MIP cannot.
4. **CSS only:** a well-tuned BP-OSD (OSD-CS, more trials, DEM permutations) matched the recorded `ilp_d` values in the tested overestimate set, often removing the need for MILP to find the CSS upper bound.

Recommended workflow for the $n \geq 108$ timeout codes. (i) run a non-early-exit MILP pass that persists `mip_dual_bound` for each attempted logical subproblem; (ii) run `QDistEvol` to obtain a tight independent U (cheap; may beat the incumbent); (iii) use U as the cutoff target for a closing MILP/SAT pass, optionally with Gurobi or `pySATDist`; (iv) report $d \in [L, U]$, exact only where every relevant subproblem is bounded and $L = U$, and flag the heuristic U as “probably tight” otherwise.

4 Limitations

The cross-check is sound but bounded: it covers $n \leq 360$ and $d \leq 24$, and the overestimate subset is low-distance. Heuristic tightness at larger (n, k, d) is an extrapolation until calibrated—on a held-out set of *certified-exact* codes one should measure the empirical probability that `QDistEvol` attains the true distance as a function of $(n, k, d, \text{budget}, \text{seed})$, and quantify its run-to-run variance. Minimum distance is NP-hard (and hard to approximate), so no method is both exact and scalable in general: beyond the regime where MIP or SAT closes, the rigorous statement is the bracket $d \in [L, U]$ with a calibrated confidence on U , not a point value. The structure of bicycle codes (circulant / group-algebra), which black-box solvers discard, is the most promising route to *certified* statements at large scale.

5 Conclusion

An independent distance package corroborated our pipeline’s recorded values on 198 codes with zero contradictions, including the up-to-12 $\times$ BP-OSD overestimate cases, and `QDistEvol` closed the BP-OSD gap against the recorded MILP value on every one of the worst codes. For larger codes, the path forward is not a magic exact oracle but a disciplined bracket: a cheap, independently-verified heuristic upper bound, persisted MILP dual bounds, and MILP/SAT closing passes for what remains closable.

Acknowledgments

This report was drafted with the assistance of Anthropic’s Claude Opus 4.8 and OpenAI GPT-5.5, which also helped implement and run the cross-check experiments described herein. All results were verified against the committed data, and the author takes responsibility for the content.

References

- [1] J. Cruz-Benito, A. W. Cross, D. Kremer, and I. Faro, *Evolutionary discovery of bivariate bicycle codes with LLM-guided search*, arXiv:2606.02418 (2026).
- [2] M. Webster, A. Jacob, and O. Higgott, *Distance algorithms for classical codes, quantum codes and circuits*, arXiv:2603.22532 (2026).
- [3] A. J. Landahl, J. T. Anderson, and P. R. Rice, *Fault-tolerant quantum computing with color codes*, arXiv:1108.5738 (2011).
- [4] S. Bravyi *et al.*, *High-threshold and low-overhead fault-tolerant quantum memory*, arXiv:2308.07915 (2023).